Data Access and Integration in Python

1. Introduction

Data access and integration involve reading, combining, and processing data from multiple sources. Python provides versatile libraries such as **pandas**, **json**, and **requests** to handle different formats efficiently.

Key Concepts: - **Data Ingestion:** Collecting data from multiple sources such as files, databases, and APIs. - **Data Integration:** Combining data from different formats into a single structure for analysis. - **Best Practices:** Ensuring efficiency, consistency, accuracy, and security of data. - **Error Handling:** Anticipating and handling potential issues in data sources.

2. Reading Data from CSV Files

CSV (Comma-Separated Values) files are one of the most common formats for tabular data.

```
import pandas as pd

# Reading a CSV file
data = pd.read_csv('data.csv')
print(data.head())
```

Details and Tips: - Use `delimiter` parameter if your CSV uses other separators (e.g., `;`). - Use `encoding='utf-8'` to avoid encoding errors. - Missing values can be handled with `data.fillna()` or `data.dropna()`. - For large files, consider reading in chunks using the `chunksize` parameter.

3. Reading Data from Excel Files

Excel files are widely used for structured data with multiple sheets.

```
# Reading an Excel file
excel_data = pd.read_excel('data.xlsx', sheet_name='Sheet1')
print(excel_data.head())
```

Details and Tips: - Specify the correct sheet name to read data. - Use `engine='openpyxl'` for `.xlsx` files. - You can read multiple sheets by passing `sheet_name=None` to get a dictionary of DataFrames. - For large datasets, read selected columns with the `usecols` parameter.

4. Reading Data from JSON Files

JSON (JavaScript Object Notation) is a common format for structured and nested data.

```
import json

# Reading JSON file
with open('data.json', 'r') as file:
    json_data = json.load(file)

print(json_data)
```

Details and Tips: - Nested JSON can be flattened using `pd.json_normalize()`. - Validate JSON structure to prevent parsing errors. - When reading JSON from a URL, use `requests.get(url).json()`.

5. Reading Data from APIs

APIs provide programmatic access to remote datasets.

```
import requests

response = requests.get('https://api.example.com/data')
data = response.json()

print(data)
```

Details and Tips: - Check for rate limits and required authentication. - Use headers and API keys securely. - Handle errors with `try-except` blocks and check `response.status_code`. - Convert API response to pandas DataFrame for easier analysis.

6. Best Practices for Data Ingestion

1. **Validate Input Data:** Ensure data types, formats, and structures are consistent.
2. **Handle Missing and Corrupted Data:** Clean or impute missing values appropriately.
3. **Efficient Libraries:** Use pandas or NumPy for large datasets.
4. **Consistent Encoding:** Specify encoding to avoid errors when reading files.
5. **Logging and Monitoring:** Keep logs of ingestion processes for reproducibility.
6. **Secure API Access:** Use HTTPS and securely store API keys.
7. **Batch Processing:** For large files or streaming data, read in batches or chunks.
8. **Data Profiling:** Analyze new datasets for anomalies and inconsistencies.
9. **Documentation:** Keep metadata about sources, transformations, and formats.

7. Exercises

1. Read a CSV file of sales data, display the first 10 rows, and summarize sales by month.
2. Read an Excel file containing employee records and list all employees in a specific department.
3. Read a JSON file of product information and extract product names and prices.
4. Fetch data from a public API (weather, COVID-19, etc.) and display it in a DataFrame.

5. Combine data from CSV and Excel files into a single DataFrame.
6. Handle missing values in a dataset by filling with median values and verify the result.
7. Flatten a nested JSON dataset using `pd.json_normalize()`.
8. Implement error handling while reading from an API and log errors.
9. Read a large CSV file in chunks, calculate summary statistics for each chunk, and combine results.
10. Apply consistent encoding while reading multiple data files to prevent errors and verify data consistency.